



UNIVERSITÄT
LEIPZIG

9. Vorlesung Neuronale Netze - Einführung und Perceptron

INTERACTIVE VISUAL DATA MINING

Übersicht

9. Neuronale Netze - Einführung

9.1 Motivation

9.2 Vorbild Biologie

9.3 McCulloch Pitts Neuron

9.4 Perceptron

9.5 Aktivierungsfunktionen

9.6 Delta Regel

9.7 Mehrschichtige Perceptrons

9.8 Backpropagation

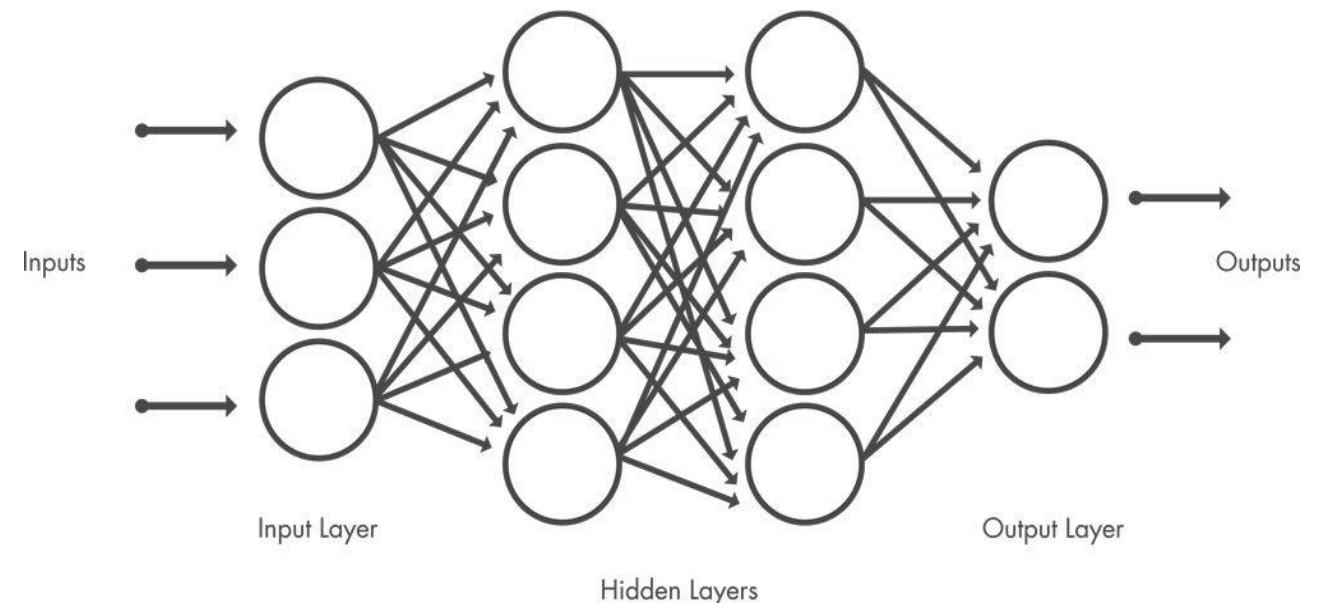
Lernziele:

- Aufbau und Funktionsweise von Neuronen und neuronalen Netzen verstehen
- McCulloch-Pitts Neuron und Perceptron erklären können
- Delta-Regel verstehen und anwenden
- Verstehen wie Mehrschicht-Perceptrons komplexere Probleme ermöglichen
- Den Backpropagation Algorithmus verstehen und anwenden können

NEURONALE NETZE

- Künstliche neuronale Netze sind:
 - Massiv parallel verbundene Netzwerke
 - Bestehend aus einfachen (adaptiven) Elementen (Neuronen)
 - Organisiert in hierarchischen Strukturen (Schichten)

- Diese Netze sollen in derselben Art wie biologische Nervensysteme mit der Welt interagieren. (Kohonen 84)
- Informationsverarbeitung erfolgt durch viele einfache Einheiten
- Komplexes Verhalten entsteht durch deren Zusammenspiel



NEURONALE NETZE

– Forschung:

- Modellierung und Simulation biologischer neuronaler Netze
- Speicherung von Information
- Funktionsapproximation
- ...

– Anwendungen:

- Analyse Sensordaten
- Medizin
- Schrifterkennung
- Bilderkennung
- Zeitreihenanalyse
- Sprachverarbeitung
-

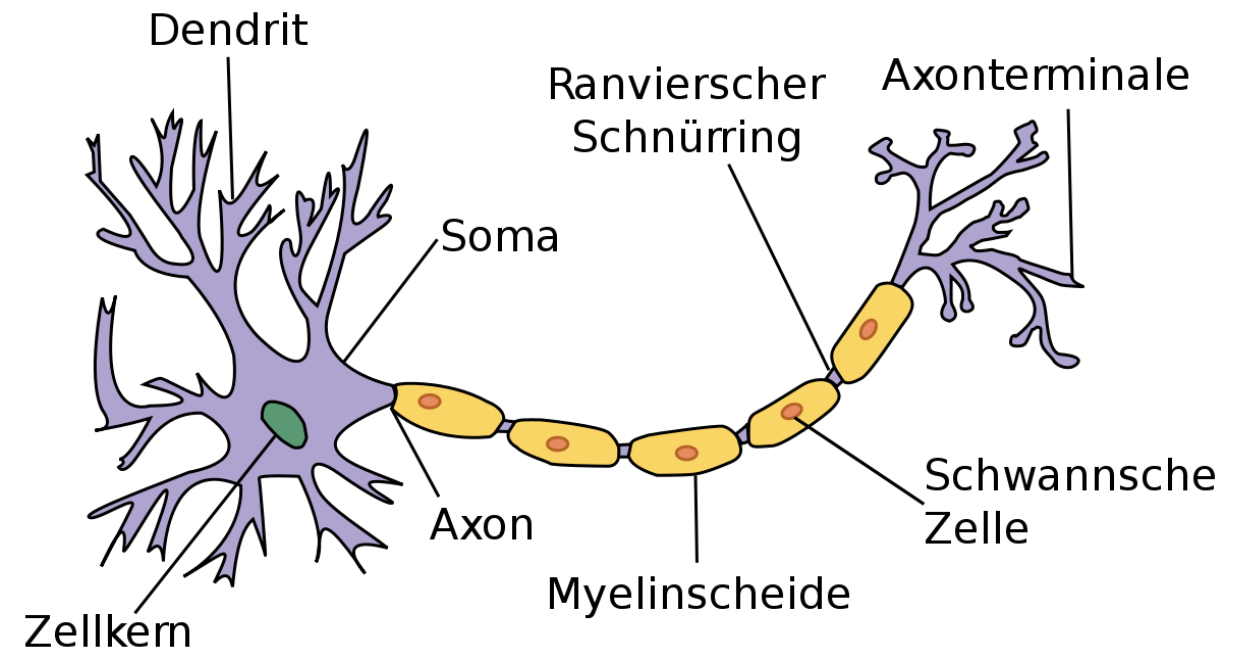
NEURONALE NETZE

- Künstliche neuronale Netze sind gekennzeichnet durch:
 - Massiv parallele Informationsverarbeitung
 - Propagierung der Informationen durch Kanten
 - Verteilte Informationsspeicherung
 - Black Box Modell
- Phasen:
 - Aufbauphase (Topologie des Netzes)
 - Trainingsphase (Lernen)
 - Arbeitsphase (Propagation)

NEURONALE NETZE

– Vorbild Biologie:

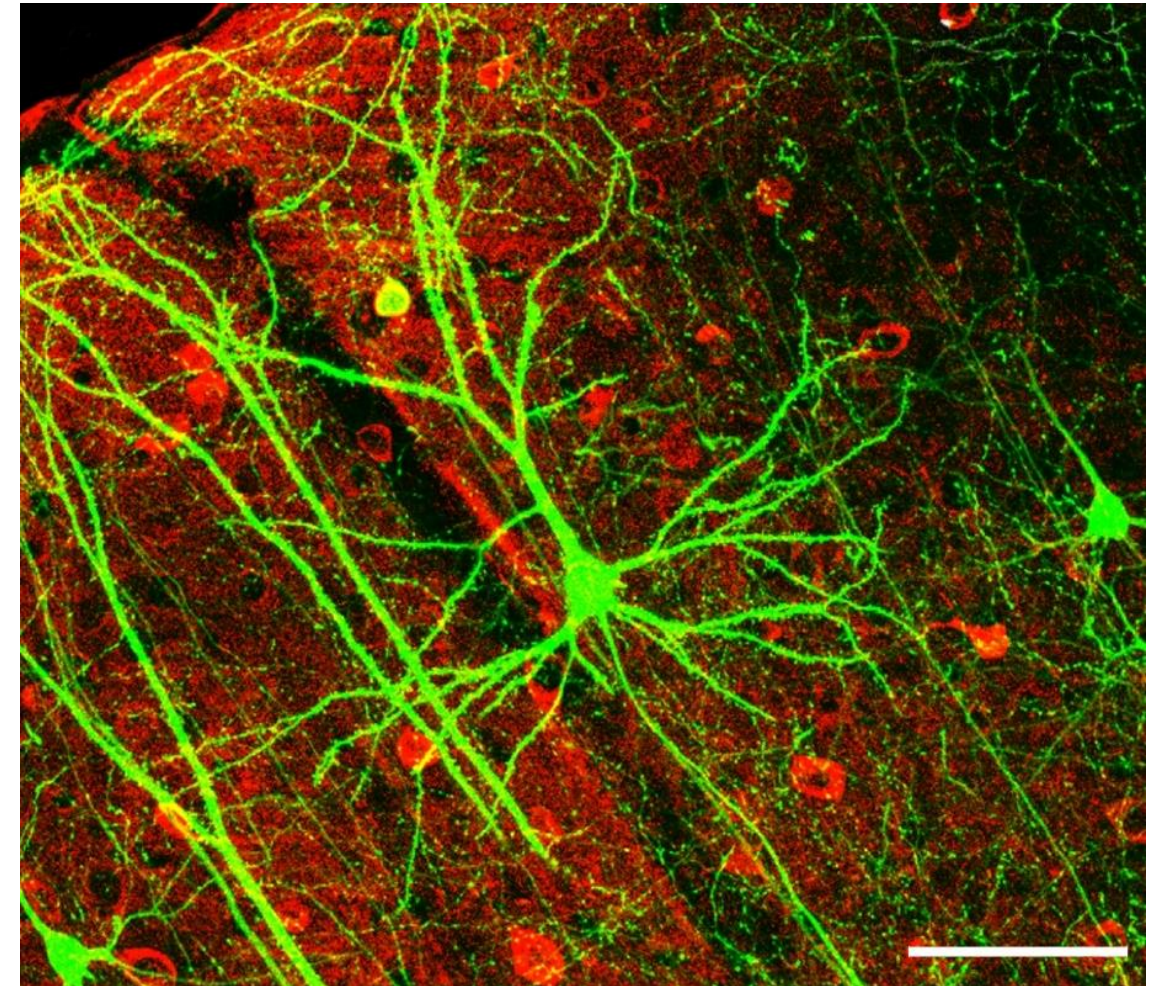
- Neuronen können
 - Informationen aufnehmen (Dendriten)
 - Informationen verarbeiten (Zellkörper)
 - Informationen weiterleiten (Axon, Synapsen)



NEURONALE NETZE

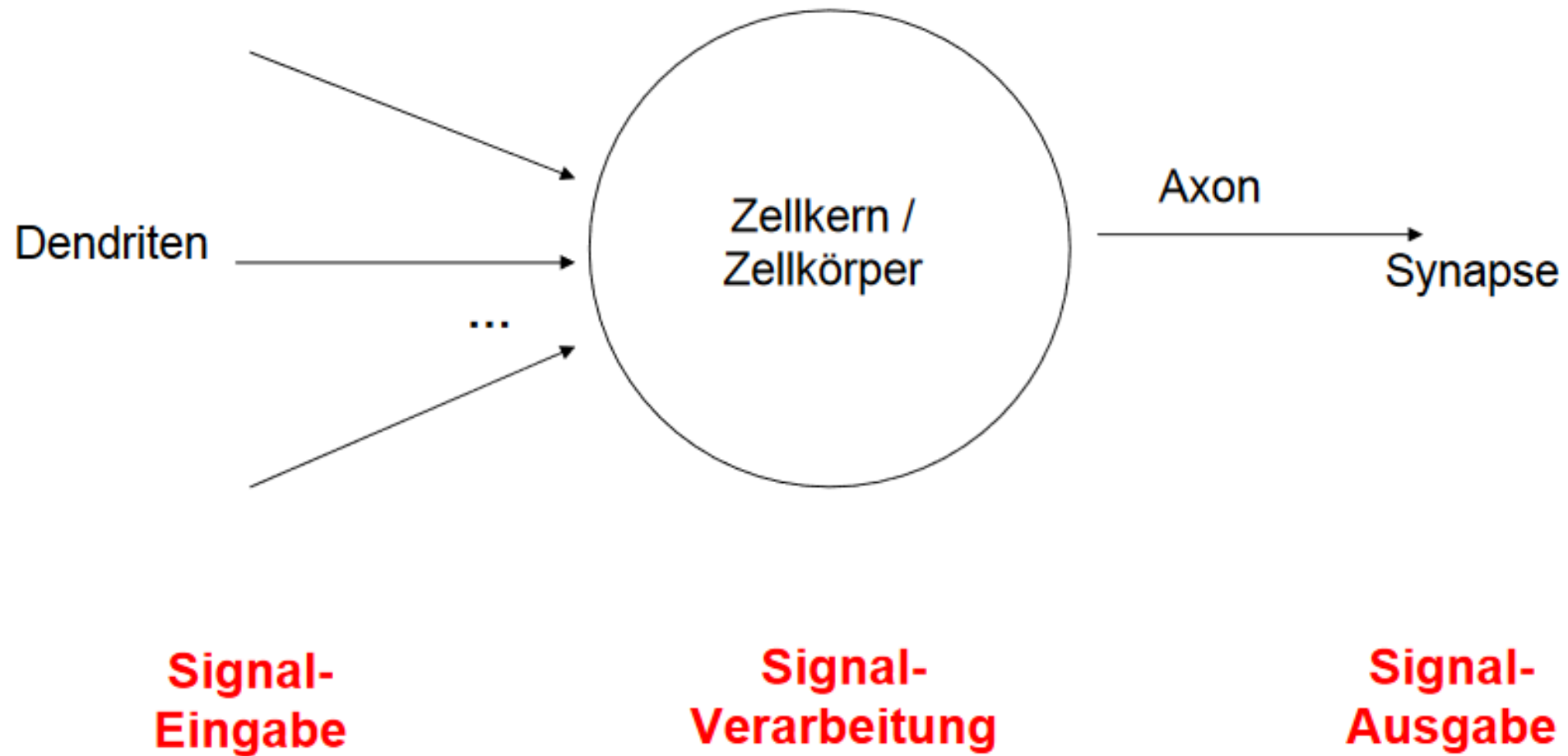
Vorbild Biologie

- Das menschliche Gehirn besteht aus
 - ca. 10^{11} Neuronen, die mit
 - ca. 10^4 anderen Neuronen durch je
 - ca. 10^{14} bis 10^{15} Synapsen verbunden sind



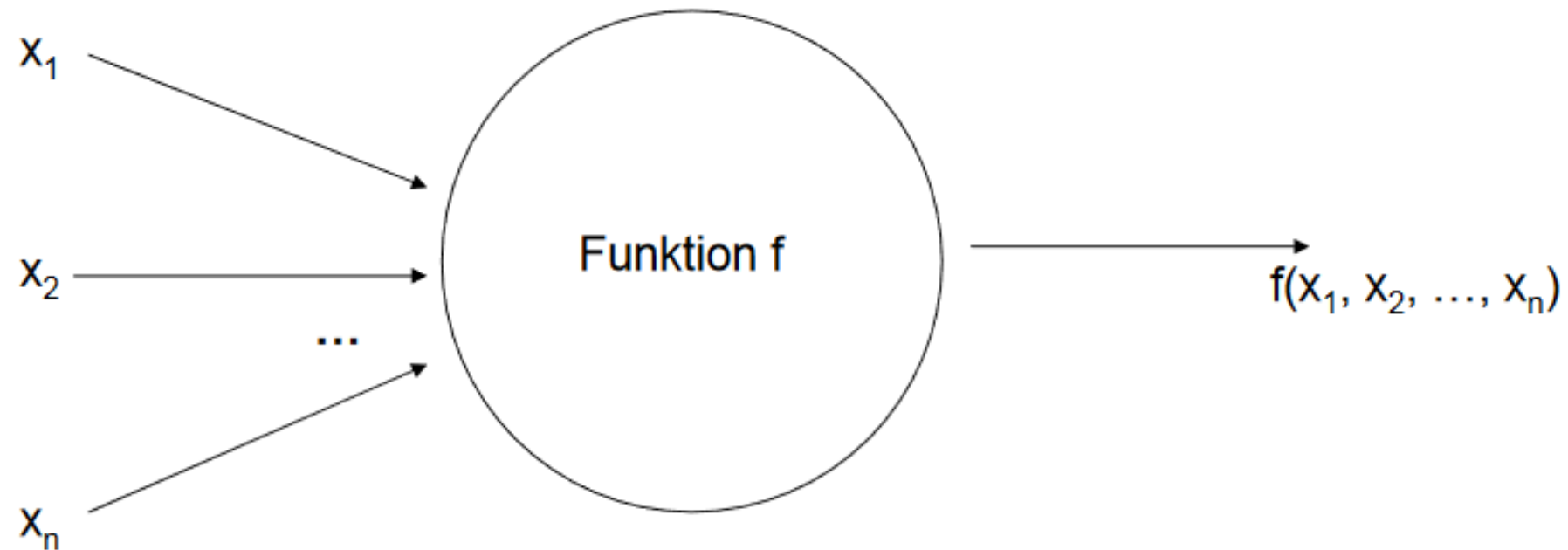
NEURONALE NETZE

- Abstraktion



NEURONALE NETZE

– Modell



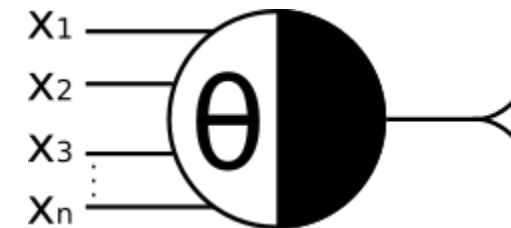
McCulloch-Pitts-Neuron 1943:

$$x_i \in \{0, 1\} =: \mathbb{B}$$

$$f: \mathbb{B}^n \rightarrow \mathbb{B}$$

NEURONALE NETZE

- 1943: Warren McCulloch & Walter Pitts
- Beschreibung neurologischer Netzwerke
- Modell McCulloch Pitts Neuron
- Grundidee:
 - Neuron ist entweder aktiv oder inaktiv
 - Fähigkeiten entstehen durch Vernetzung der Neuronen
- Es werden nur statische Netze betrachtet
- Topologie wird vorher festgelegt
- Es werden keine neuen Verbindungen beim Lernen erstellt
 - Widerspruch zur Biologie!

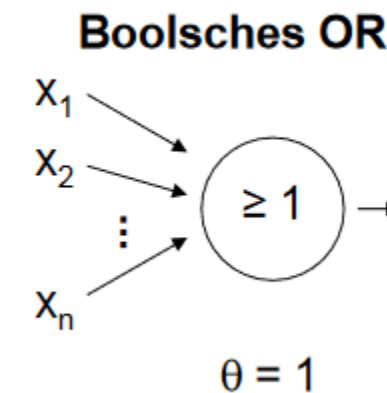
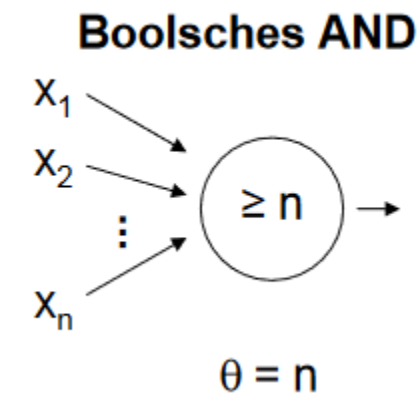


NEURONALE NETZE

- McCulloch Pitts Neuron
 - n binäre Eingangssignale x_1 bis x_n
 - Schwellwert $\theta > 0$

$$f(x_1, \dots, x_n) = \begin{cases} 1 & \text{falls } \sum_{i=1}^n x_i \geq \theta \\ 0 & \text{sonst} \end{cases}$$

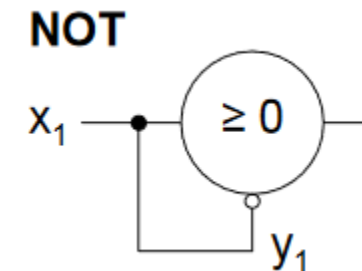
- Damit sind folgende logische Funktionen realisierbar:



NEURONALE NETZE

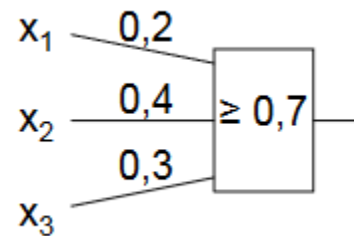
- McCulloch Pitts Neuron
 - n binäre Eingangssignale x_1 bis x_n
 - Schwellwert $\theta > 0$
 - Zusätzlich m binäre hemmende Signale y_1 bis y_m
- Ist ein $y_j = 1$, dann ist die Ausgabe 0
- Sonst gilt:
 - Summe der Eingänge $\geq$ Schwellwert $\rightarrow$ Ausgabe = 1
 - Ansonsten Ausgabe = 0

$$\tilde{f}(x_1, \dots, x_n; y_1, \dots, y_m) = f(x_1, \dots, x_n) \cdot \prod_{j=1}^m (1 - y_j)$$

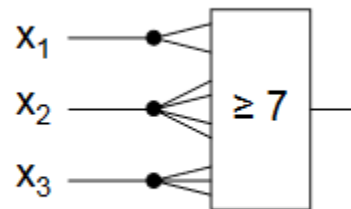


NEURONALE NETZE

- Verallgemeinerung mit Gewichten



- Ist äquivalent zu



- Dieser Knoten liefert 1 bei

$$0,2 x_1 + 0,4 x_2 + 0,3 x_3 \geq 0,7$$

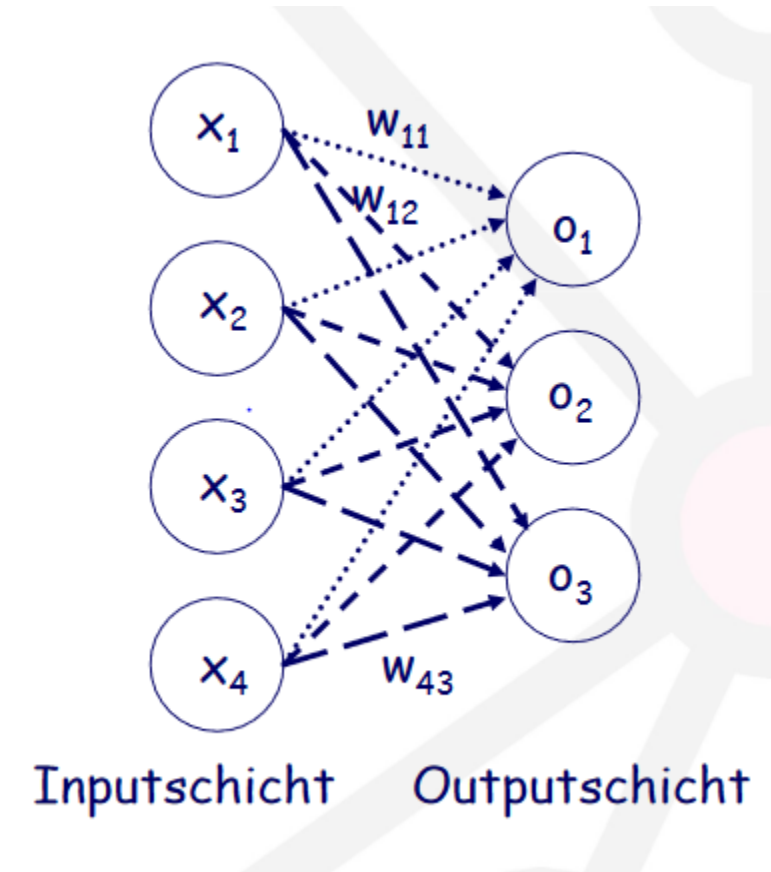
- Wenn man die Gewichte mit 10 multipliziert

$$2 x_1 + 4 x_2 + 3 x_3 \geq 7$$

Die Eingänge entsprechend dupliziert, erhält man einen äquivalenten Knoten.

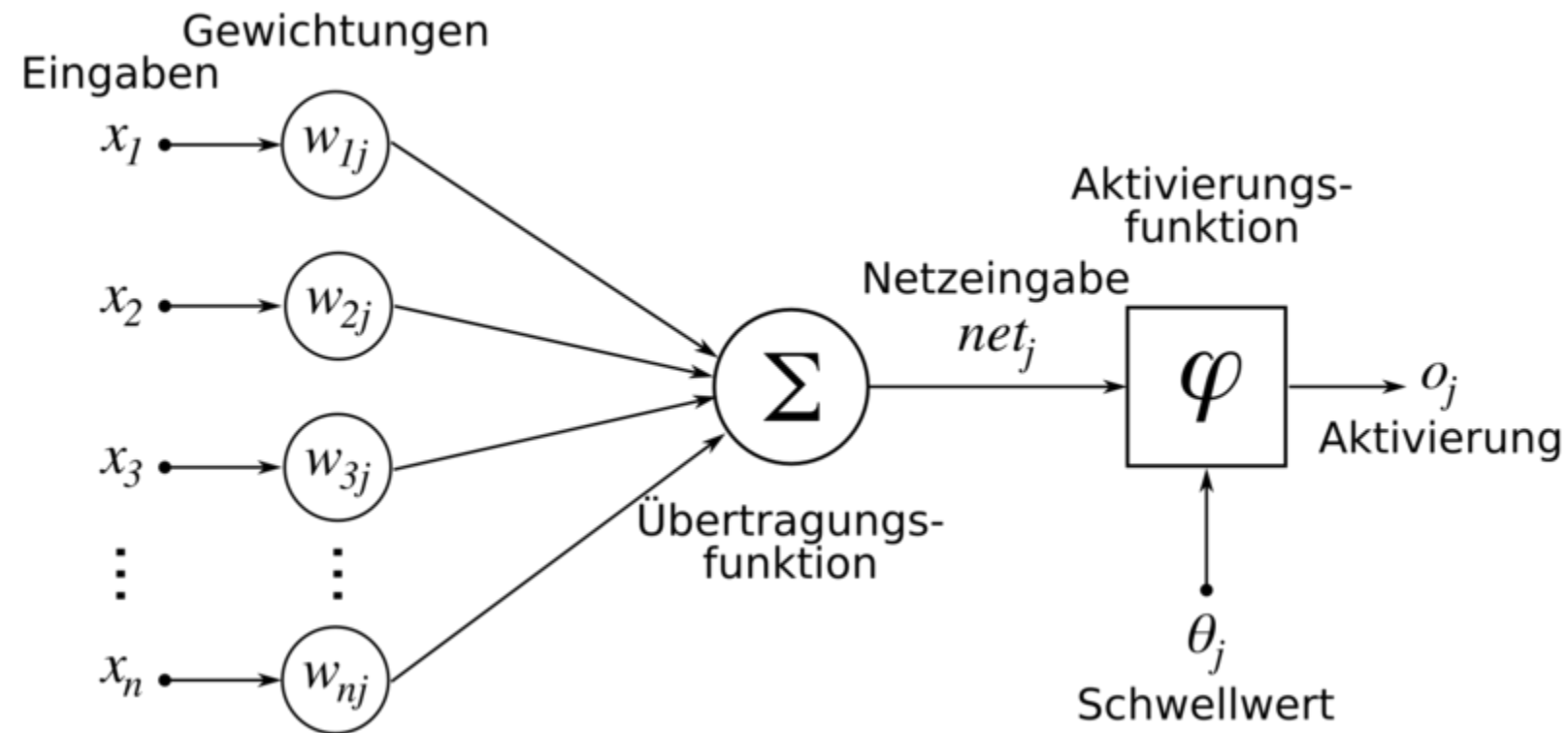
NEURONALE NETZE

- Perceptron (Rosenblatt 1958)
 - Jedes Outputneuron hat einen eigenen unabhängigen Netzbereich
 - Zur Vereinfachung hat jedes Netz im folgenden nur einen Output
 - War historisch als Hardwareimplementierung gedacht
 - Später auf Multilayer Perceptrons erweitert



NEURONALE NETZE

– Genereller Aufbau



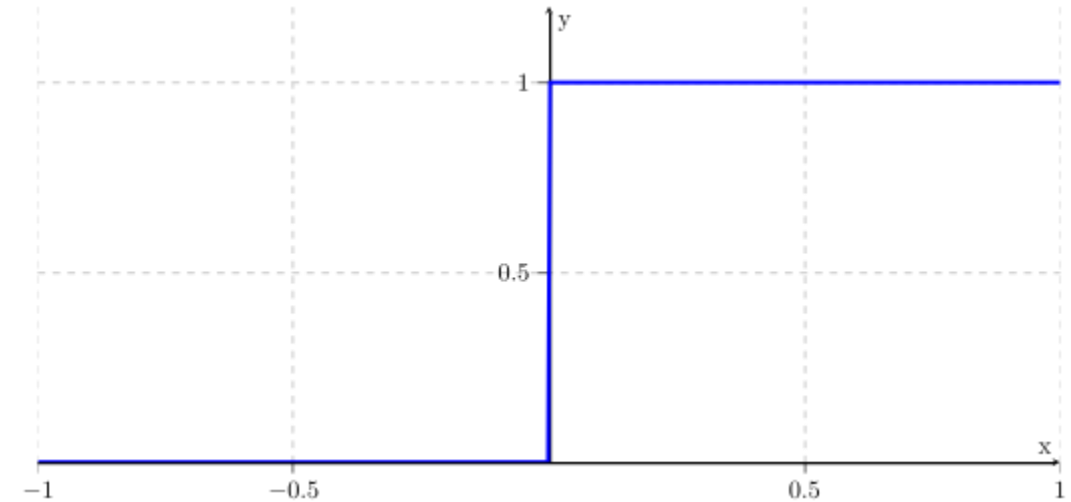
NEURONALE NETZE

- Aktivierungsfunktionen
 - Funktionen, die bestimmen, ob und wie stark ein Neuron „aktiv“ wird
 - Sie werden auf die gewichtete Summe der Eingaben angewendet
 - Sie entscheiden, was das Neuron weitergibt
- Ohne Aktivierungsfunktion wäre ein neuronales Netz nur eine lineare Funktion und könnte keine komplexen Muster lernen
- Mit Aktivierungsfunktion können nichtlineare Zusammenhänge gelernt werden

NEURONALE NETZE

- Aktivierungsfunktionen
 - Schwellwertfunktion:
 - Nimmt nur die Werte 0 und 1 an
 - Schwellwert bestimmt die Aktivierung
 - Alles oder Nichts Funktionsweise

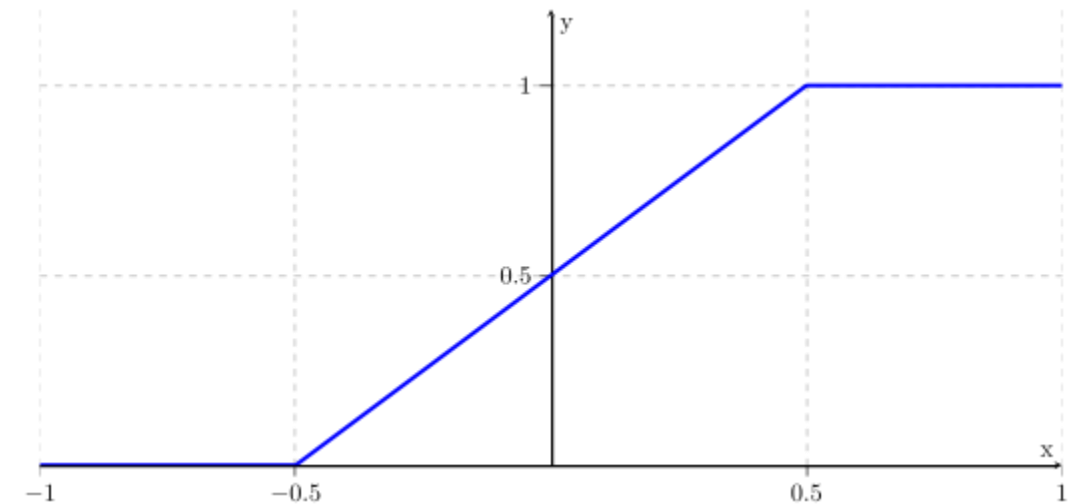
$$\phi(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$



NEURONALE NETZE

- Aktivierungsfunktionen
- Stückweise lineare Funktion:
 - Abbildung eines begrenzten Intervalls mit einer linearen Funktion
 - Außerhalb konstante Werte

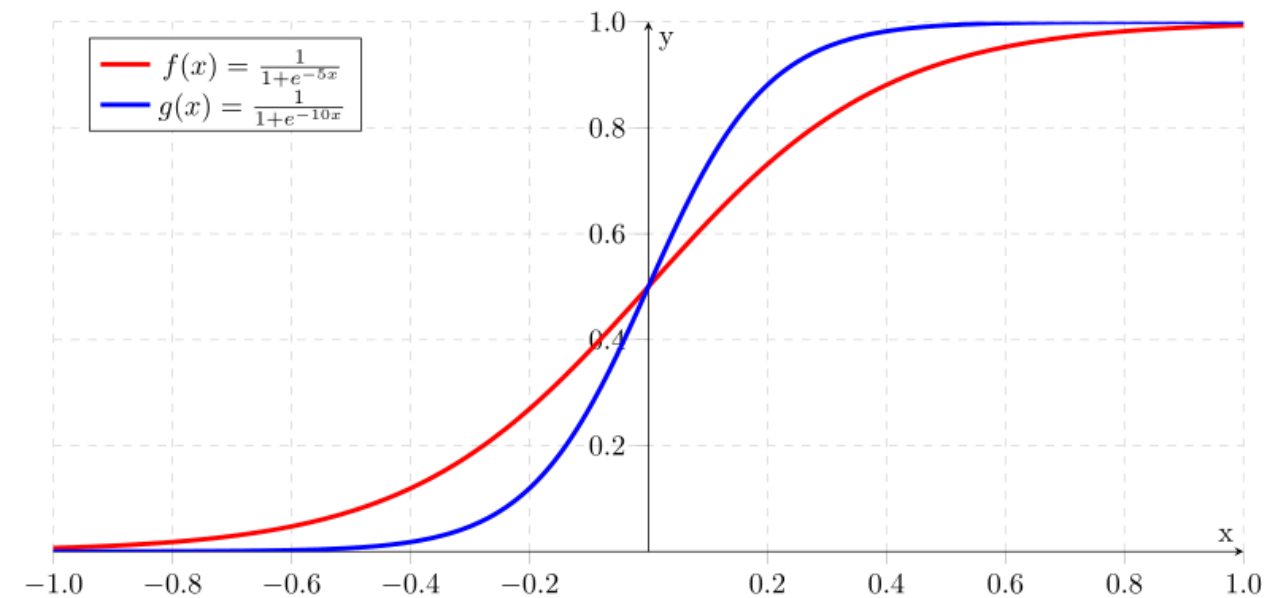
$$\phi(x) = \begin{cases} 0, & x \leq -\frac{1}{2} \\ x + \frac{1}{2}, & -\frac{1}{2} < x < \frac{1}{2} \\ 1, & x \geq \frac{1}{2} \end{cases}$$



NEURONALE NETZE

- Aktivierungsfunktionen
 - Sigmoid Funktion
 - Wurden lange als Standardfunktion genutzt
 - Steigungsmaß alpha kann modifiziert werden
 - Vanishing Gradient Problem
 - Ableitung in Sättigung $\approx 0 \rightarrow$ Gradient verschwindet

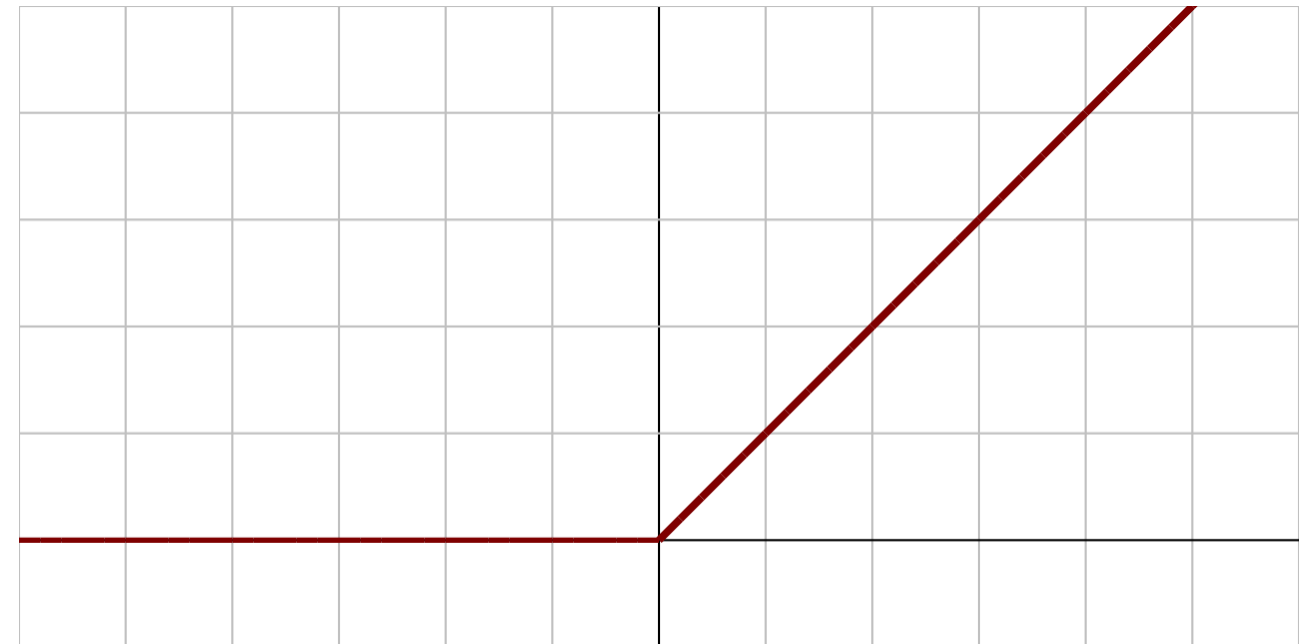
$$\phi(x) = \frac{1}{1 + e^{-\alpha x}}$$



NEURONALE NETZE

- Aktivierungsfunktionen
 - Rectifier (ReLU) Funktion
 - Rectified Linear Activation Unit
 - Abwandlung der stückweisen linearen Funktion
 - Nur positiver Teil wird linear abgebildet

$$\phi(x) = \max(0, x)$$



NEURONALE NETZE

- Outputneuron hat:
 - Schwellwert s
 - Aktivität a aus den Inputs und Gewichten
 - Nutzt Aktivierungsfunktion, um den Output zu berechnen
 - Historisch wurde die Sprungfunktion verwendet
 - Diese ist aber nicht differenzierbar →
gradientenbasiertes Lernen (Delta-Regel)
braucht eine differenzierbare Funktion

NEURONALE NETZE

- Die Sprungfunktion bildet eine lineare Trenngerade zwischen 2 Klassen

$$w_1 x_1 + w_2 x_2 \geq \theta \begin{cases} \text{J} \rightarrow 1 \\ \text{N} \rightarrow 0 \end{cases}$$

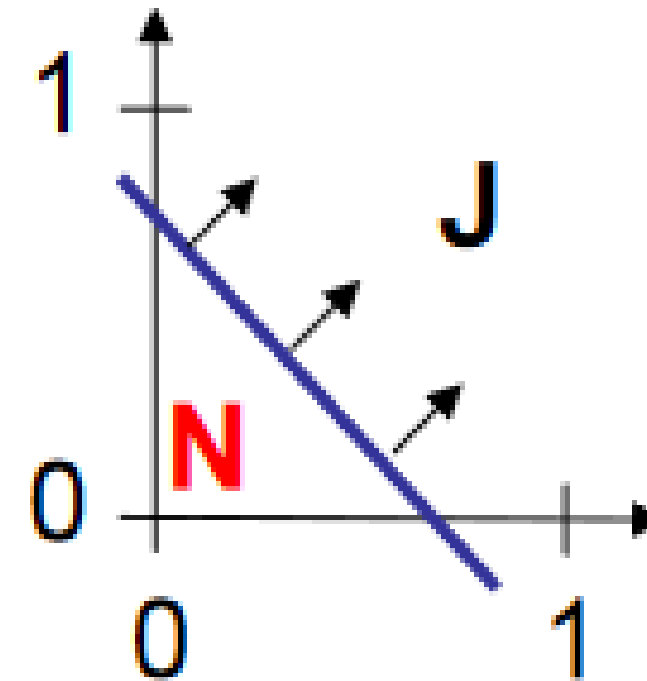
- Umgestellt nach x_2

$$x_2 \geq \frac{\theta}{w_2} - \frac{w_1}{w_2} x_1 \begin{cases} \text{J} \rightarrow 1 \\ \text{N} \rightarrow 0 \end{cases}$$

- Beispiel:

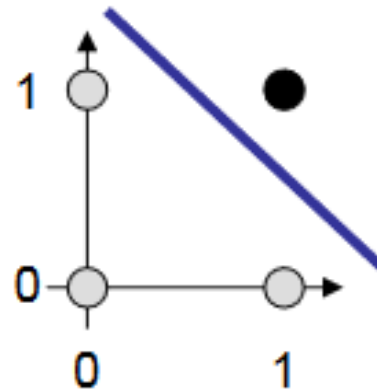
$$0.9 x_1 + 0.8 x_2 \geq 0.6$$

$$x_2 \geq 3/4 - 9/8 x_1$$

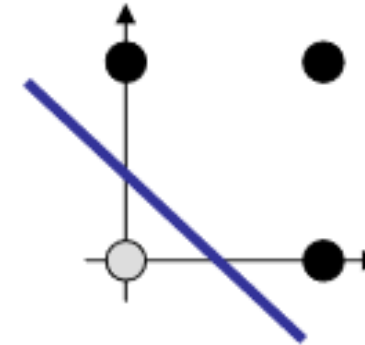


NEURONALE NETZE

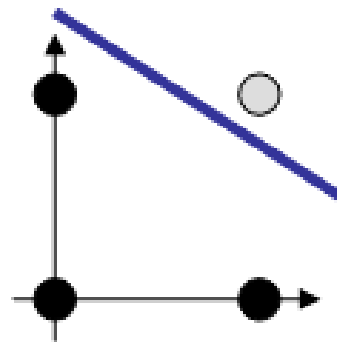
AND



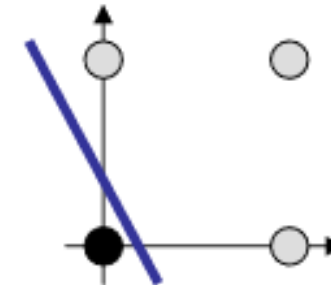
OR



NAND



NOR



NEURONALE NETZE

- Lernverfahren Delta Regel
 - Beim Training werden die Daten dem Netz als Input gezeigt
 - Output für die Beispiele bekannt (supervised)
 - Vergleich von Soll und Ist Zustand im Output
- Bei Abweichung werden Schwellwert und Gewichte adaptiert
- Ist nur auf differenzierbare Aktivierungsfunktionen anwendbar!
- Anpassung kann nach jedem Input oder nach jeder Epoche durchgeführt werden
- Epoche = Ein Durchlauf aller Trainingsdaten

NEURONALE NETZE

- Lernverfahren Delta Regel

$$\Delta w_{ji} = \alpha (t_j - y_j) \phi'(h_j) x_i$$

- Alpha: konstante Lernrate
- ϕ ist die Aktivierungsfunktion
- ϕ' ist die Ableitung von ϕ
- t_j ist die Sollausgabe

- y_j ist die Istaussgabe

- h_j ist die gewichtete Summe der Inputs des Neurons

- x_i ist der Input i

- Mit

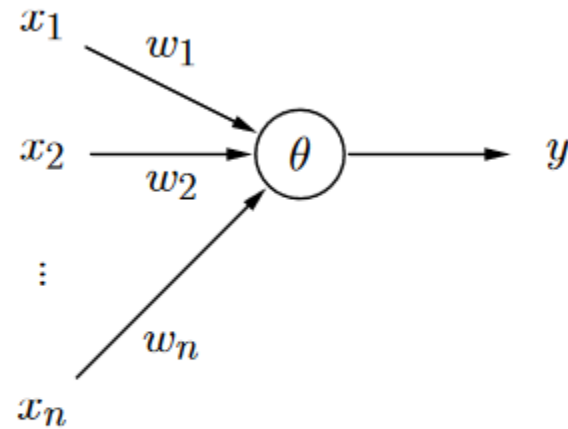
$$h_j = \sum_i w_{ji} x_i$$

$$y_j = \phi(h_j)$$

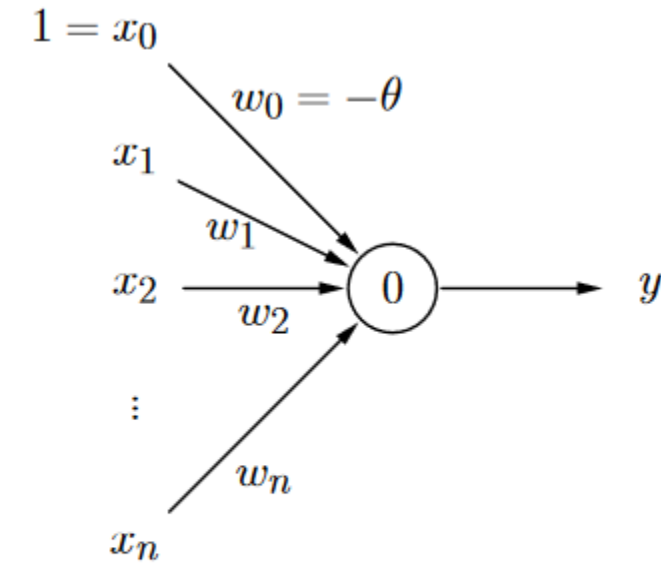
- Die gleiche Formel wird auf den Schwellwert angewendet

NEURONALE NETZE

- Lernverfahren Delta Regel
- Der Schwellwert ist damit auch nur ein Gewicht. Wird oft Bias genannt.



$$\sum_{i=1}^n w_i x_i \geq \theta$$



$$\sum_{i=1}^n w_i x_i - \theta \geq 0$$

NEURONALE NETZE

- Lernverfahren Delta Regel
- Initialer Schwellwert -1
- Lernfaktor 0.2

Eingang1	Eingang2	Ausgang
0	0	0
0	1	1
1	0	0
1	1	1

	$i_1 = 0/i_2 = 0$	$i_1 = 0/i_2 = 1$	$i_1 = 1/i_2 = 0$	$i_1 = 1/i_2 = 1$
ω_0	1.0	1.0	1.2	1.2
ω_1	1.0	1.0	0.8	0.8
ω_2	1.0	1.0	1.0	1.0

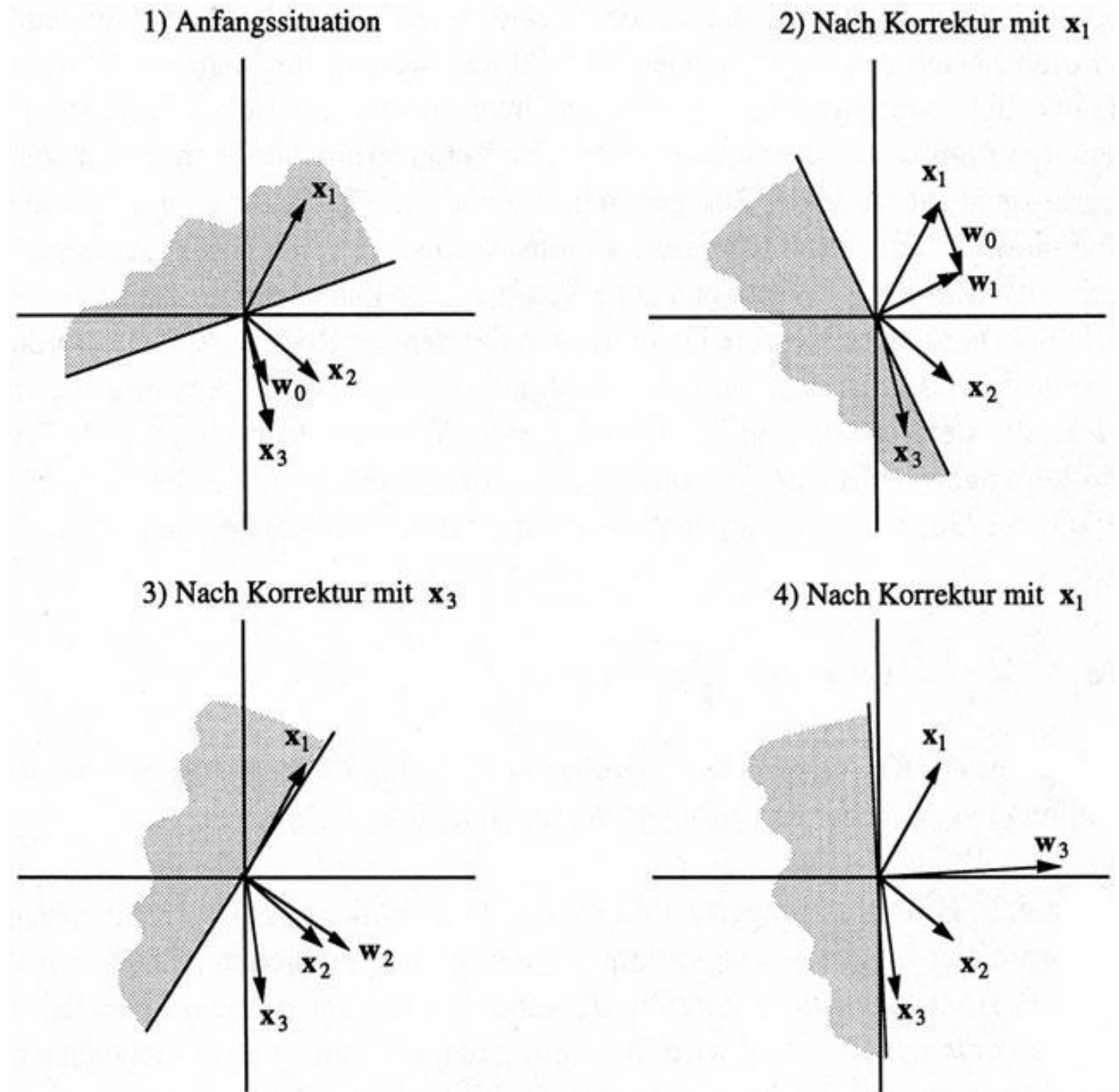
	$i_1 = 0/i_2 = 0$	$i_1 = 0/i_2 = 1$	$i_1 = 1/i_2 = 0$	$i_1 = 1/i_2 = 1$
ω_0	1.2	1.0	1.0	1.0
ω_1	0.8	0.8	0.8	0.8
ω_2	1.0	1.2	1.2	1.2

NEURONALE NETZE

- Lernverfahren Delta Regel
 - Konvergenz und Korrektheit
 - Satz: Wenn ein Perzeptron eine Klasseneinteilung lernen kann, dann lernt es diese mit der Delta Regel in endlich vielen Schritten
- Problem: Falls das Perzeptron das Modell nicht erlernt, kann man nicht entscheiden:
 - Ob genügend Epochen gelernt wurde oder
 - Das Problem nicht erlernbar ist
- → Es gibt keine obere Schranke für die Lerndauer
 - → Overfitting Problematik

NEURONALE NETZE

- Lernverfahren Delta Regel
- Geometrische Interpretation nach Rojas

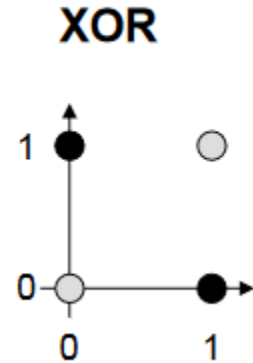


NEURONALE NETZE

- Lernverfahren
 - Gegeben ist ein Trainingsdatensatz
 - Daten sind disjunkt in 2 Menge A und B aufgeteilt
 - Gesucht wird eine trennende Hyperebene
 - Basiert auf den Gewichten
 - Teilt A und B auf
- Problem: A und B müssen linear trennbar sein!

NEURONALE NETZE

– XOR Problem



x_1	x_2	xor
0	0	0
0	1	1
1	0	1
1	1	0

$$w_1 x_1 + w_2 x_2 \geq \theta$$

$$\Rightarrow 0 < \theta$$

$$\Rightarrow w_2 \geq \theta$$

$$\Rightarrow w_1 \geq \theta$$

$$\Rightarrow w_1 + w_2 < \theta$$

$$w_1, w_2 \geq \theta > 0$$

$$\Rightarrow w_1 + w_2 \geq 2\theta$$

Widerspruch!

- Triviales Problem ist mit einem einzelnen Neuron (einschichtigem Perzeptron) nicht lösbar!

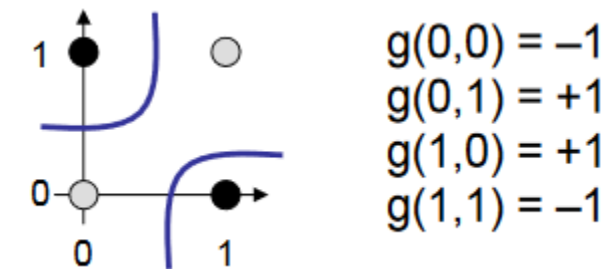
NEURONALE NETZE

– XOR Problem

- 1969 publiziert von Minsky und Papert
- Folgerung (damalige Lesart): künstliche Neuronen sind eine Sackgasse
- Forschung wurde daraufhin ca. 15 Jahre eingestellt

– Lösungen sind:

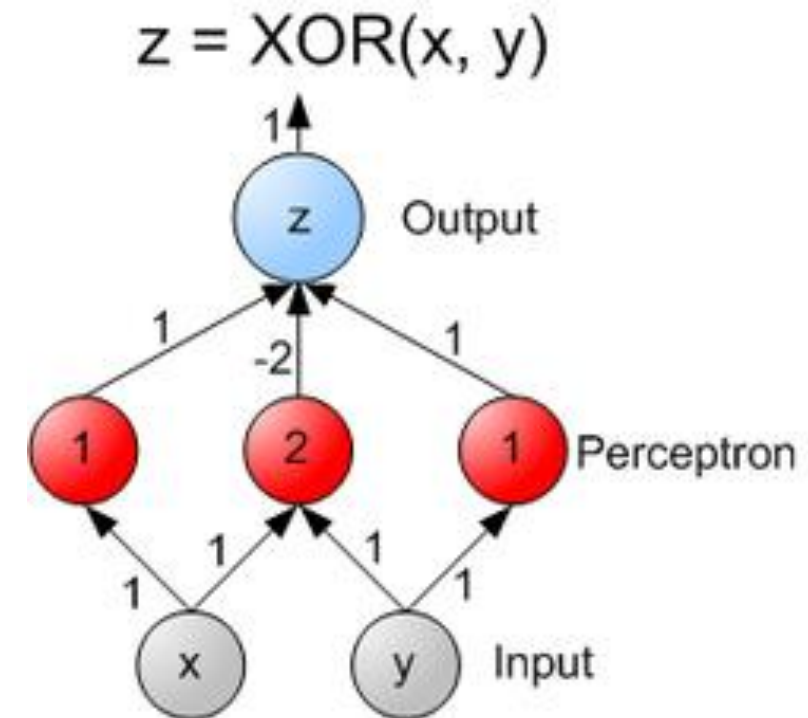
- Mehrschichtige Perzeptrons (Feedforward Netze)
- Nichtlineare Trennfunktionen



$$g(x_1, x_2) = 2x_1 + 2x_2 - 4x_1x_2 - 1 \quad \text{mit} \quad \theta = 0$$

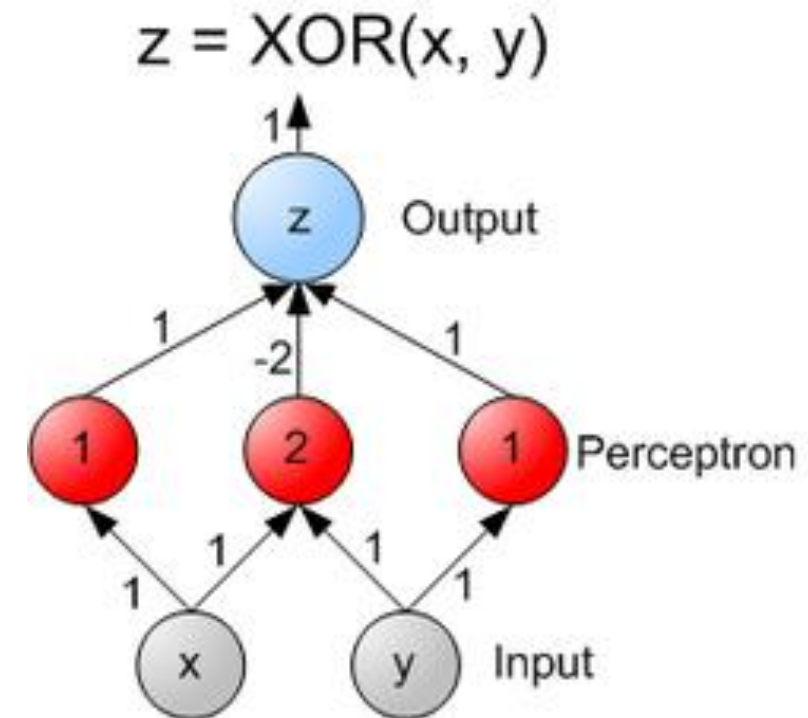
NEURONALE NETZE

- Mehrschichtige Perceptrons
- Einführung von zusätzlichen Schichten
 - Hidden layer
- Signal wird vom Eingang zu den Ausgängen weitergeleitet
- Feedforward Netze



NEURONALE NETZE

- Topologien
 - Fully connected
 - Alle Neuronen einer Schicht werden mit allen Neuronen der folgenden Schicht verbunden
 - Short Cuts
 - Einige Neuronen sind zudem mit Neuronen in weiter entfernten Schichten verbunden



NEURONALE NETZE

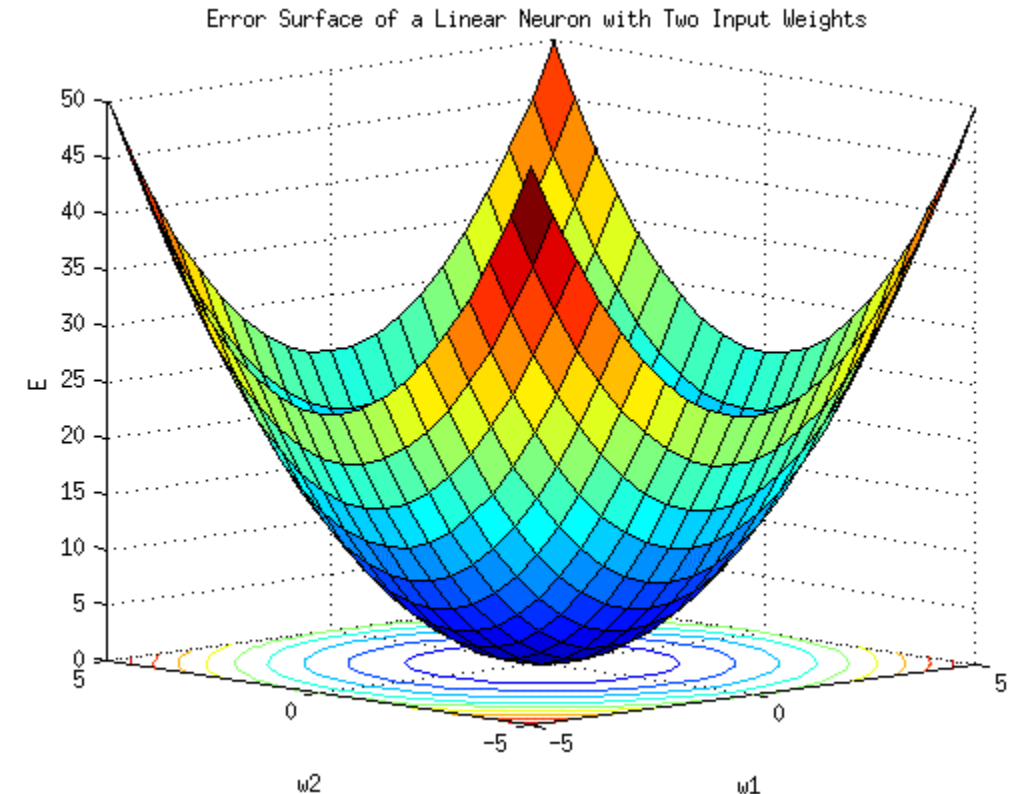
- Lernverfahren
 - Delta Regel berechnet Gradient auf einer Schicht
 - Fehler verteilt sich aber auf mehrere Schichten
 - Gewichte müssen nun über mehrere Layer angepasst werden
 - Wie verteilt man den Einfluss des Fehlers auf die Gewichte?
- Idee: Propagiere den Fehler durch das Netz
 - Von den Ausgängen zu den Eingängen
 - Trainingsdaten werden vorwärts durch das Netz propagiert
 - Fehler werden rückwärts durch das Netz propagiert
 - *Backpropagation (1986)*

NEURONALE NETZE

- Backpropagation Ablauf:
 - Trainingsdaten werden durch das Netz propagiert
 - Ausgabe des Netzes wird evaluiert, die Differenz zwischen Soll und Ist wird als Fehler angenommen
 - Der Fehler wird von der Ausgabe zum Eingang propagiert
 - Die Gewichte der Verbindungen, die zu diesem Fehler geführt haben, werden angepasst
- Anpassung des Netzes auf den falsch klassifizierten Datensatz
- Dabei muss verhindert werden, dass das Netz sich überspezifisch an einzelnen Datenpunkten orientiert
- Nutzung einer Lernrate, um die Anpassung der Gewichte zu steuern

NEURONALE NETZE

- Backpropagation
- Spezialfall des Gradientenverfahrens
 - Meist mittlerer quadratischer Fehler als Fehlerfunktion
 - Ausgang der Neuronen basiert auf den Gewichten und Eingaben
 - Bilden ein elliptisches Paraboloid
- Backpropagation sucht nach dem stärksten Abstieg im Gradientenverfahren

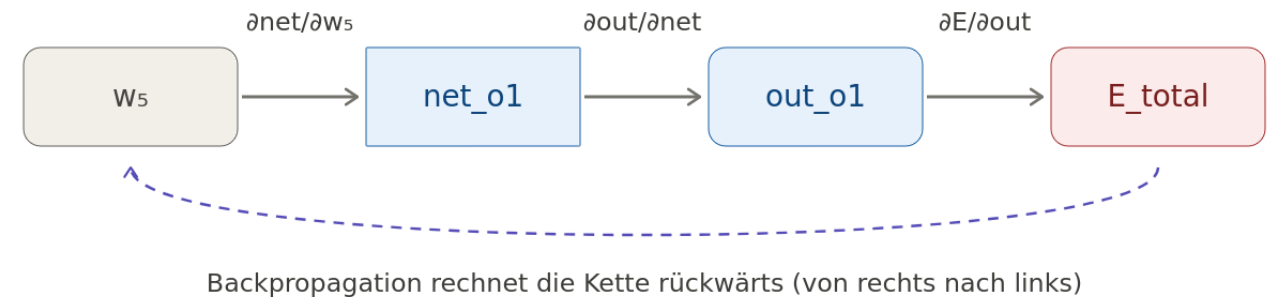


NEURONALE NETZE

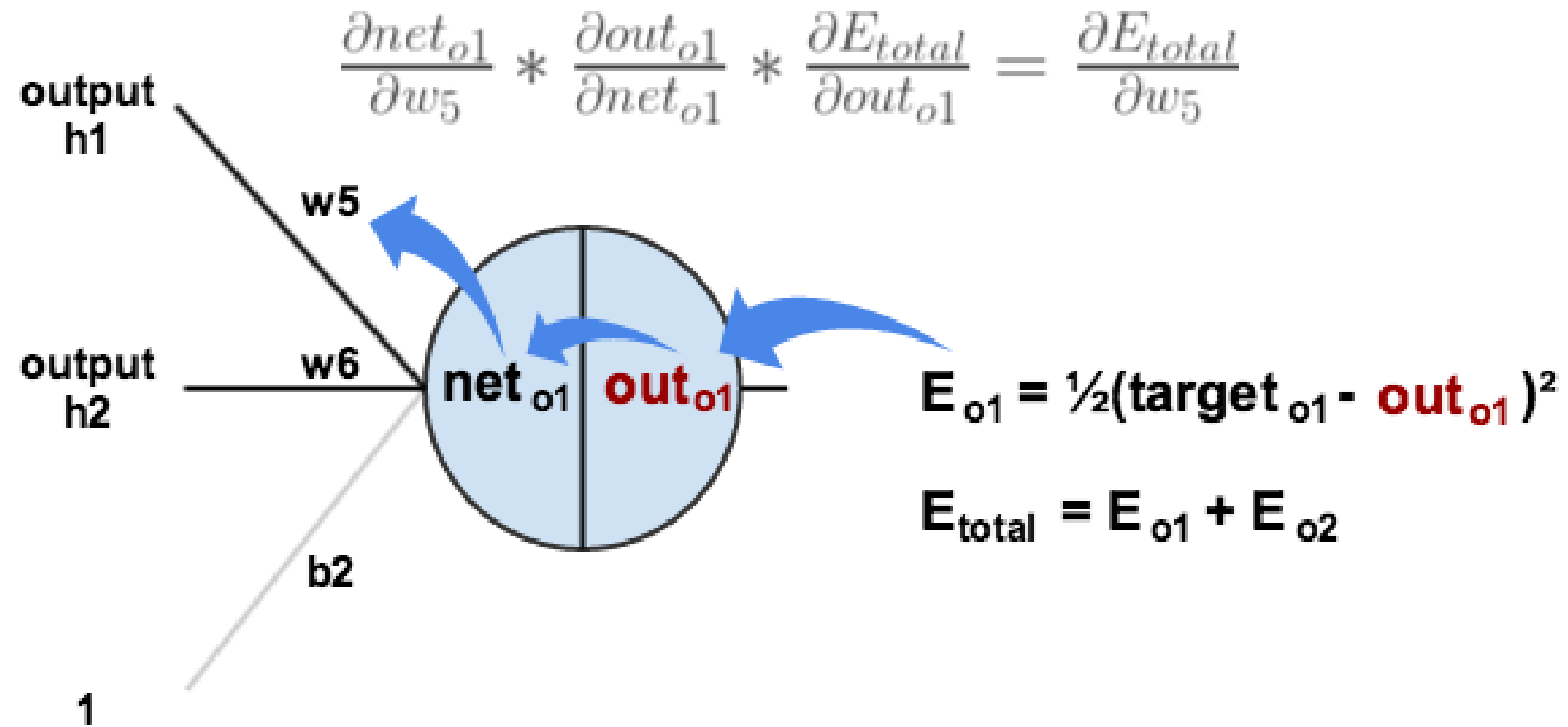
- Backpropagation
 - Berechnung des Einflusses eines Gewichtes w auf den Gesamtfehler
 - Zum Beispiel w_5 : $\frac{\partial E_{total}}{\partial w_5}$
 - Liest sich: „die partielle Ableitung von E_{total} nach w_5 “
 - Oder: „Der Gradient in Bezug auf w_5 “

- Partielle Ableitung können mit der Kettenregel berechnet werden:

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$

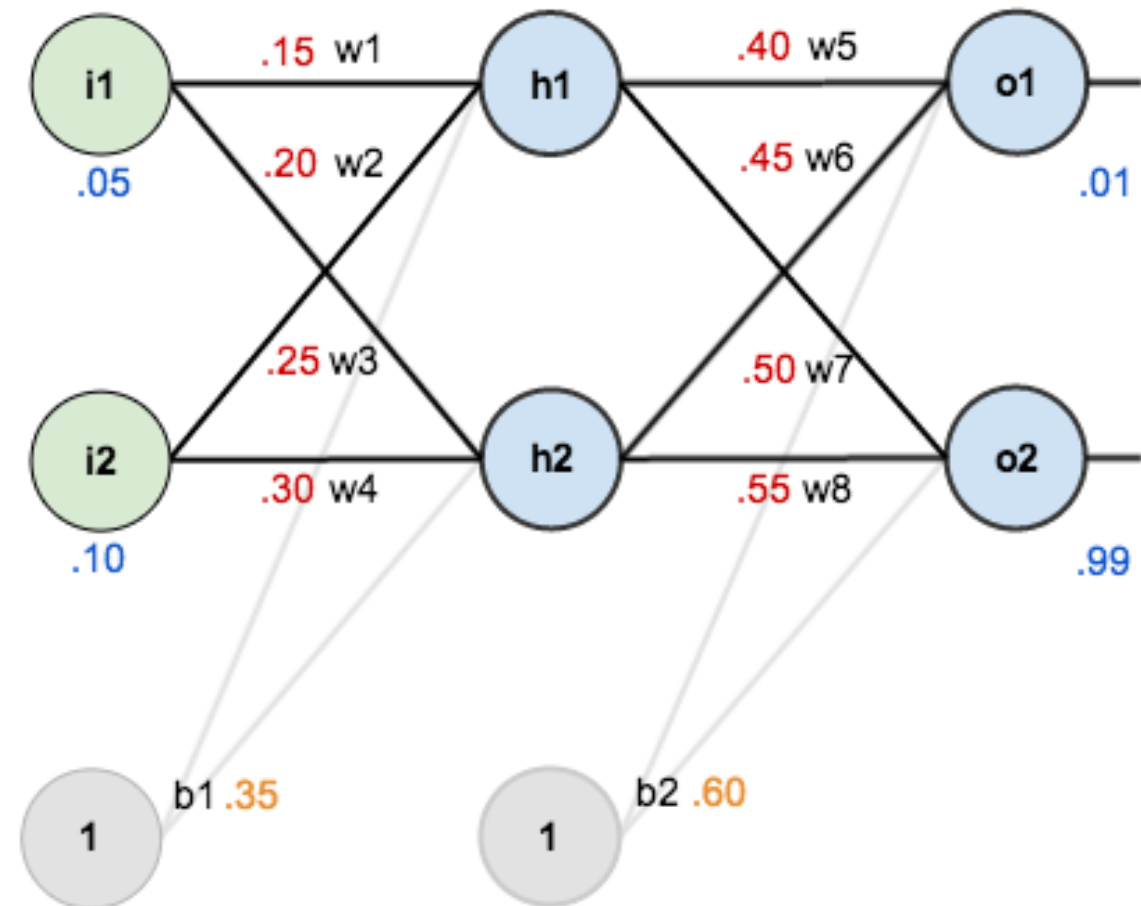


NEURONALE NETZE



NEURONALE NETZE

- Beispiel:
- Für die Inputs 0.05 und 0.10 sollen die Ausgaben 0.01 und 0.99 gelernt werden
- Als Aktivierungsfunktion wird die Logistische Funktion (Sigmoid) genutzt
- 0.05 wird durch das Netz propagiert
- o_1 gibt ~ 0.7514 aus
- o_2 gibt ~ 0.7729 aus



NEURONALE NETZE

- Beispiel:
- Berechnung des Quadratischen Fehlers

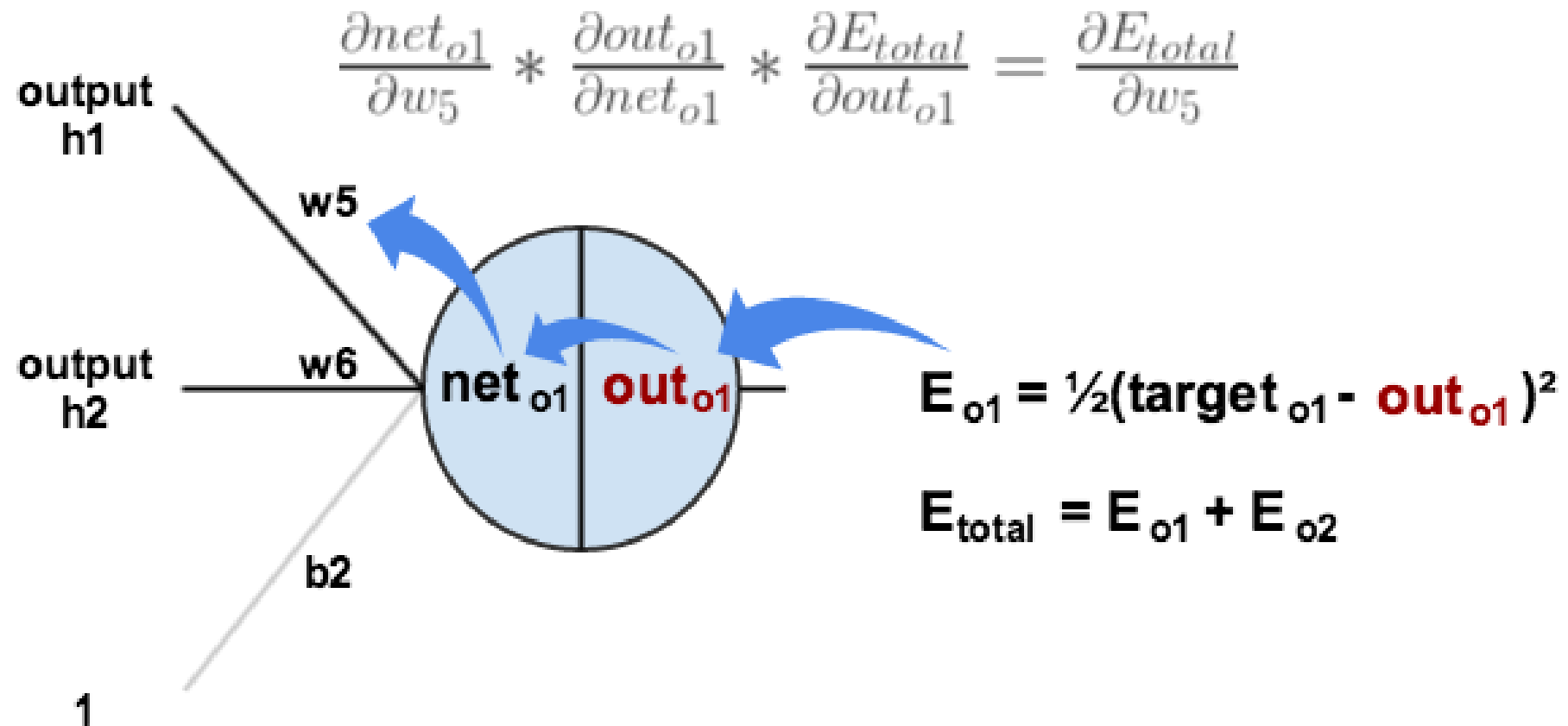
$$E_{total} = \sum \frac{1}{2}(target - output)^2$$

$$E_{o1} = \frac{1}{2}(target_{o1} - out_{o1})^2 = \frac{1}{2}(0.01 - 0.75136507)^2 = 0.274811083$$

$$E_{o2} = 0.023560026$$

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

NEURONALE NETZE



NEURONALE NETZE

- Beispiel

- E_{total}

$$E_{\text{total}} = \frac{1}{2}(target_{o1} - out_{o1})^2 + \frac{1}{2}(target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{\text{total}}}{\partial out_{o1}} = 2 * \frac{1}{2}(target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{\text{total}}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

NEURONALE NETZE

- Beispiel
- Die partielle Ableitung der logistischen Funktion (Sigmoid) ist gleich der Ausgabe multipliziert mit 1 – der Ausgabe

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1}(1 - out_{o1}) = 0.75136507(1 - 0.75136507) = 0.186815602$$

NEURONALE NETZE

- Beispiel
- Wie stark ändert sich der Wert der Übertragungsfunktion durch w_5 ?

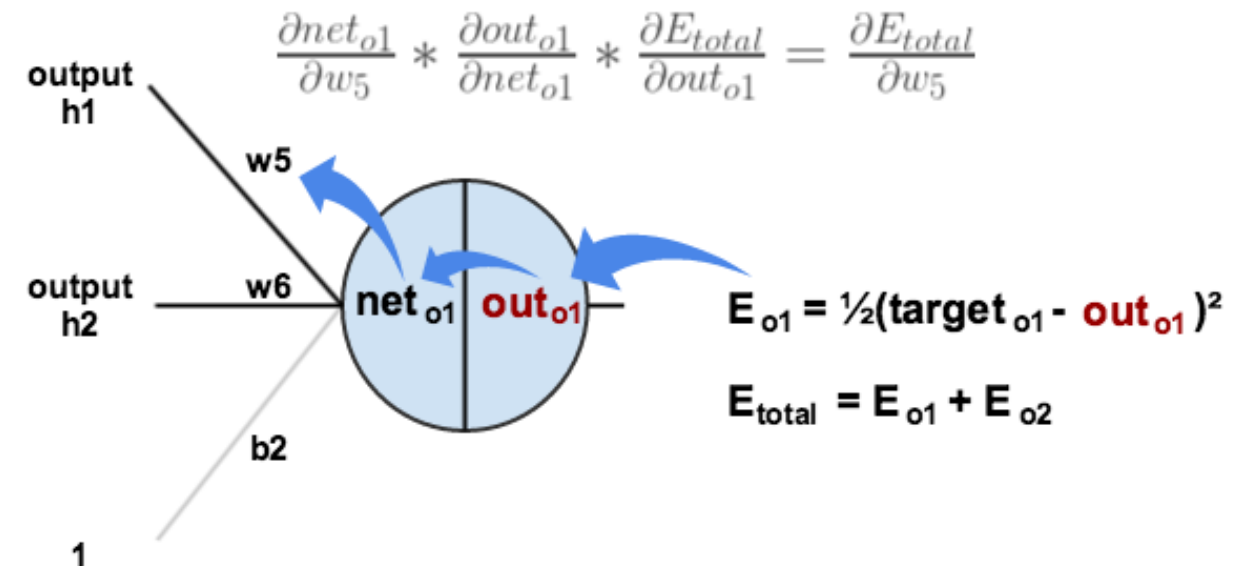
$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

NEURONALE NETZE

- Beispiel
- Nun können wir alle Teile der partiellen Ableitung berechnen:

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial w_5}$$



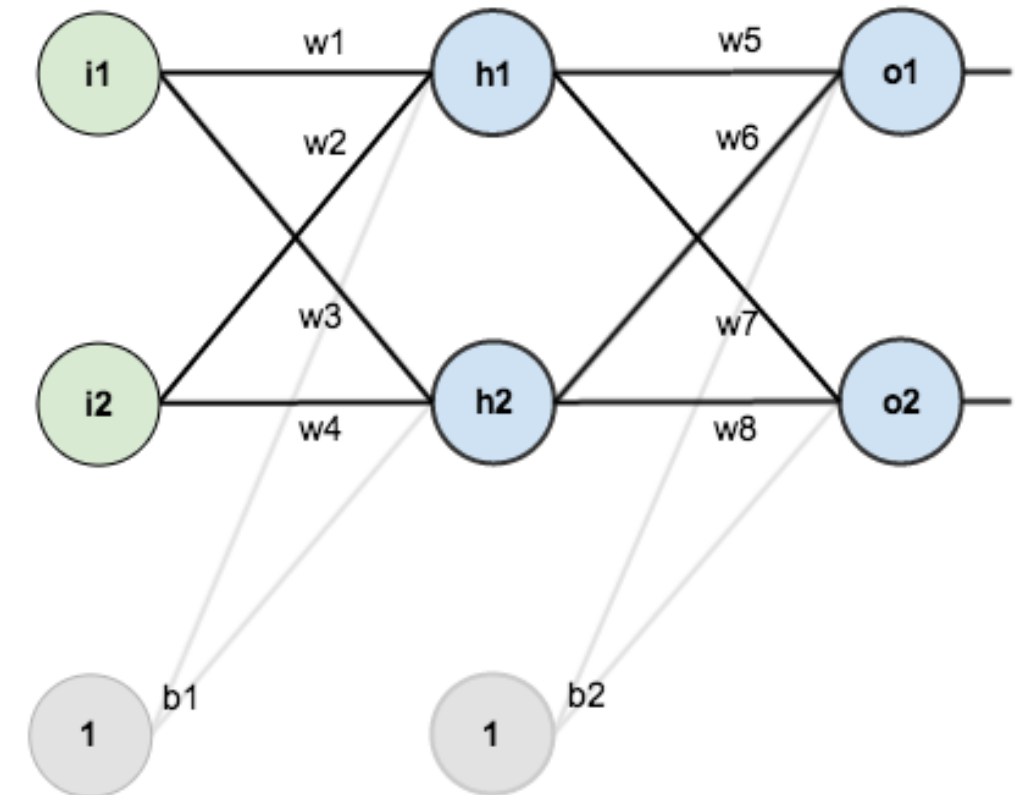
$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

- Fehlerreduktion aka Gewichts Anpassung

$$w_5^+ = w_5 - \eta * \frac{\partial E_{total}}{\partial w_5} = 0.4 - 0.5 * 0.082167041 = 0.35891648$$

NEURONALE NETZE

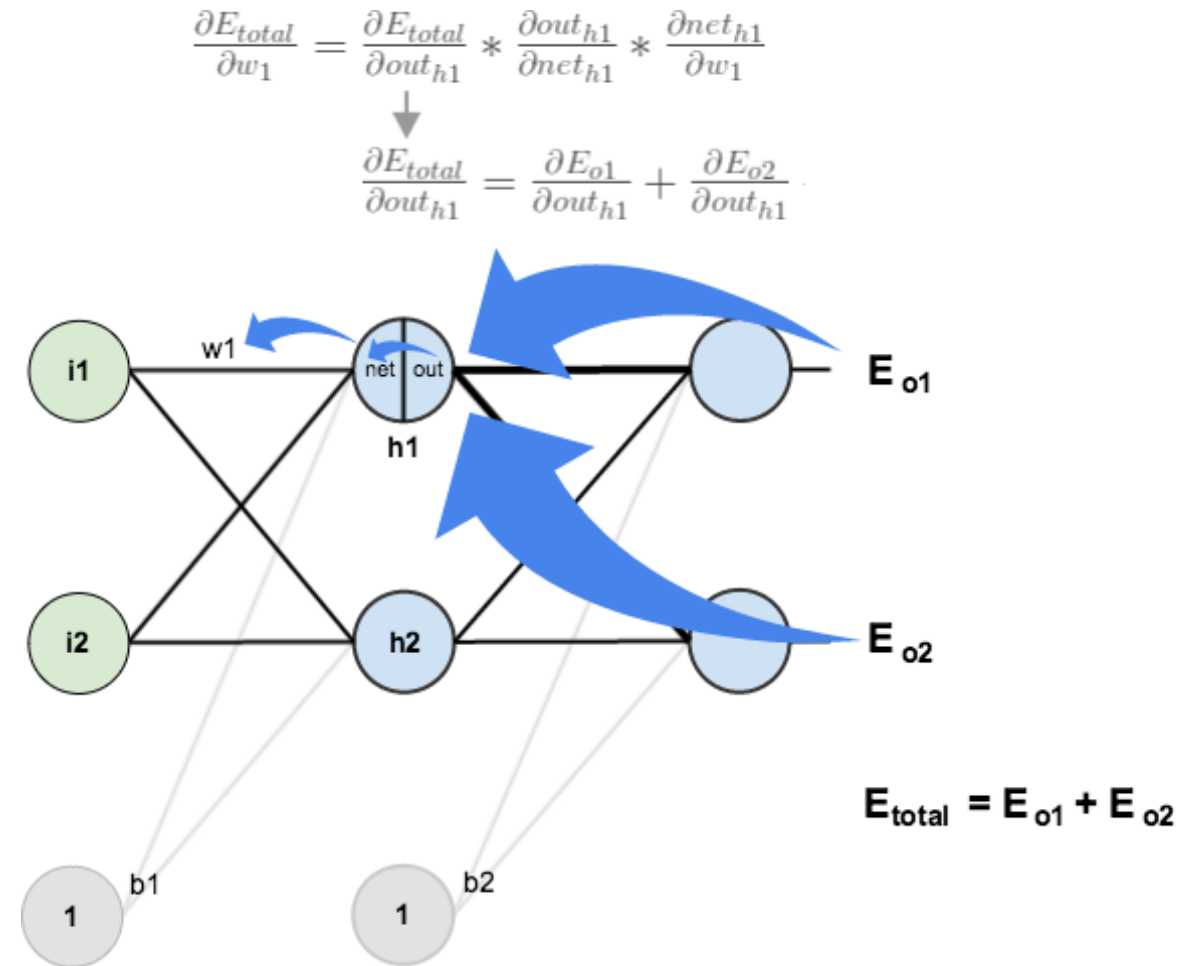
- Beispiel
- Dies wird nun für alle Gewichte auf der Ausgabeschicht berechnet
- Nachdem alle Gewichte auf dieser Schicht berechnet wurden, passt man die Gewichte der nächsten Schicht an



NEURONALE NETZE

- Beispiel
- Gewicht w_1 soll angepasst werden
- Der Fehler aus h_1 ist die Summe der Fehler in o_1 und o_2

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}}$$



NEURONALE NETZE

- Beispiel

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}}$$

- Der erste Teil wurde vorher schon berechnet!

$$\frac{\partial E_{o1}}{\partial net_{o1}} = \frac{\partial E_{o1}}{\partial out_{o1}} * \frac{\partial out_{o1}}{\partial net_{o1}} = 0.74136507 * 0.186815602 = 0.138498562$$

- $\frac{\partial net_{o1}}{\partial out_{h1}}$ ist gleich w_5

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial out_{h1}} = w_5 = 0.40$$

NEURONALE NETZE

– Beispiel

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}}$$

– Eingesetzt ergibt das nun:

$$\frac{\partial E_{o1}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial net_{o1}} * \frac{\partial net_{o1}}{\partial out_{h1}} = 0.138498562 * 0.40 = 0.055399425$$

$$\frac{\partial E_{o2}}{\partial out_{h1}} = -0.019049119$$

$$\frac{\partial E_{total}}{\partial out_{h1}} = \frac{\partial E_{o1}}{\partial out_{h1}} + \frac{\partial E_{o2}}{\partial out_{h1}} = 0.055399425 + -0.019049119 = 0.036350306$$

NEURONALE NETZE

- Beispiel

- Nun brauchen wir noch

$$\frac{\partial out_{h1}}{\partial net_{h1}} = out_{h1}(1 - out_{h1}) = 0.59326999(1 - 0.59326999) = 0.241300709$$

Und $\frac{\partial net_{h1}}{\partial w}$ für jedes Gewicht am Neuron

$$net_{h1} = w_1 * i_1 + w_3 * i_2 + b_1 * 1$$

$$\frac{\partial net_{h1}}{\partial w_1} = i_1 = 0.05$$

- Zusammen dann

$$\frac{\partial E_{total}}{\partial w_1} = \frac{\partial E_{total}}{\partial out_{h1}} * \frac{\partial out_{h1}}{\partial net_{h1}} * \frac{\partial net_{h1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial w_1} = 0.036350306 * 0.241300709 * 0.05 = 0.000438568$$

NEURONALE NETZE

- Beispiel

- Nun kann das Gewicht angepasst werden

$$w_1^+ = w_1 - \eta * \frac{\partial E_{total}}{\partial w_1} = 0.15 - 0.5 * 0.000438568 = 0.149780716$$

- Wiederhole dies nun für alle Gewichte auf der Ebene
- Iteriere über alle Ebenen bis zu den Eingängen
- Warum läuft die Backpropagation nun von hinten?

- Die Backpropagation kann auch mittels Matrizenmultiplikation schichtweise berechnet werden
- Großes Beschleunigungspotential durch GPUs und FPGAs

NEURONALE NETZE

- Durch Backpropagation in Feedforward Netzen können auch nicht lineare Funktionen approximiert werden
- Lernerfolg ist aber nicht garantiert
 - Kann sehr lange Trainingszeiten benötigen
 - Topologie des Netzes hat Auswirkung auf den Lernerfolg
- Weitere Mechanismen wie Pooling und Konvolution helfen bei der Beschleunigung und verbessern die Ergebnisse
- Komplexere Netztopologien können zu besseren Ergebnissen führen

NEURONALE NETZE

Ausblick: vom MLP zu Deep Learning

- **Universelle Approximation:** schon ein MLP mit einer hinreichend großen Hidden-Schicht kann beliebige stetige Funktionen annähern
- **Warum dann Tiefe?** Tiefe Netze lernen Merkmale hierarchisch (Kanten → Formen → Objekte) – das ist oft viel effizienter als eine breite Schicht
- **Der Haken:** ein MLP nimmt keine Struktur in den Daten an (z. B. Pixelnachbarschaft oder Wortreihenfolge)

Wohin führt das?

- Bilder → CNN: Faltung + Pooling für räumliche Muster
- Sequenzen → RNN → LSTM → Transformer: von Schritt-für-Schritt über langes Gedächtnis (Gates) zu Attention
- Ohne Labels → Autoencoder: komprimierte Repräsentationen, Anomalieerkennung, generative Basis
- XAI: erklären, was diese Blackboxes gelernt haben
- Fundament bleibt: MLP + Backpropagation